

Changing RDM: the traceable loop

Document ID	AGENT_WORKFLOW
Revision	395af08
Date	2026-10-03
Status	Controlled in git

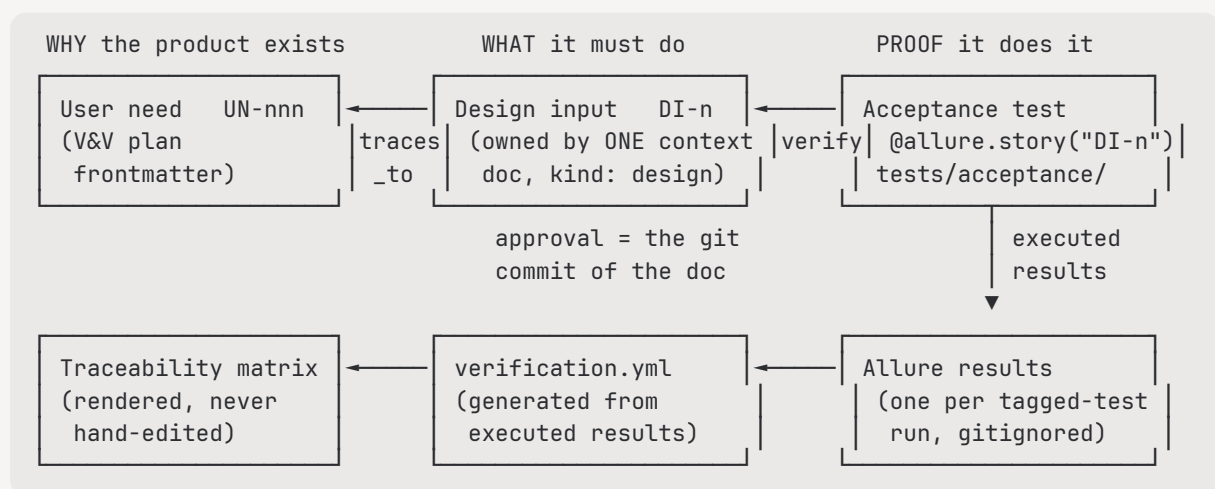
Contents

1	The intent — what a “complete, traceable implementation” means	3
2	Do I need a design input?	4
3	The loop	4
3.1	Step 0 — see the landscape	4
3.2	Step 1 — the user need (the WHY)	4
3.3	Step 2 — declare the design input (the WHAT)	5
3.4	Step 3 — commit the design docs FIRST (the approval)	5
3.5	Step 4 — implement (the HOW)	5
3.6	Step 5 — make the test real (the PROOF)	5
3.7	Step 6 — run the gates as CI will	6
3.8	Step 7 — commit, push, PR (the independent review)	7
4	Before a release — the chain review	7
5	A worked example — from this repository’s own history	7
6	Hard rules	8
7	When a gate fails	9

Audience: anyone — human or AI agent — about to change RDM. **Promise:** follow this once, top to bottom, and your change passes CI's design-controls pipeline on the first try, with a complete evidence chain behind it.

1 The intent – what a “complete, traceable implementation” means

RDM is a design-controls tool for medical-device software, and it governs its own development with the same controls. The goal of any change here is **not just working code** — it is working code plus an unbroken, machine-checkable chain of evidence:



Whether the test actually *means* anything is judged by a human: the change lands through a pull request that a reviewer other than the author approves.

Every arrow is checked by a gate. A change is **complete** when:

1. its behavior is stated as a **design input** (DI-*n*) tracing to a **user need** (UN-*nnn*) — so anyone can ask *why does this code exist?* and get an answer;
2. the design doc declaring it is **committed before the implementation** — the commit is the approval; there is no separate sign-off bureaucracy;
3. a test tagged @allure.story("DI-*n*") **passes** — executed evidence, not a claim;
4. the change is merged through a **pull request approved by a reviewer other than the author**, who judges whether the test actually verifies the requirement — because a test can pass without proving anything, and an agent grading its own homework reliably over-grades;
5. the traceability matrix regenerates cleanly — it is derived from the record, so it can never drift from reality.

Skip a link and CI fails — not as punishment, but because a missing link means someone later (an auditor, a reviewer, another agent) can no longer walk the chain. This document exists so you build the chain as *you go* instead of reverse-engineering it under a red build.

One boundary to keep in mind throughout: **Backlog tasks, GitHub issues, and plans are coordination, never evidence.** The record is only the design docs, the executed test results, and git history (docs/about/plan-vs-record.md).

2 Do I need a design input?

```
Does the change alter what RDM does (behavior, CLI, output, gate logic)?
├── YES → it is governed by a design input
│   ├── an existing DI already covers it
│   │   ├── → find it: uv run rdm story trace <DI-n | UN-nnn>
│   │   └── → skip to step 3 (edit its design doc prose if the "how" changed,
│   │       then implementation → test → PR review)
│   └── nothing covers it → full loop, step 1
└── NO (refactor, docs, comments, CI plumbing)
    └── → no DI work; commit as usual (the gates still run and should stay green)
```

3 The loop

Each step says **why** it exists, **do** exactly what, and **done when** you can verify it. The commands assume the repo root; agent sessions have dependencies synced and the local gate active automatically (session bootstrap).

3.1 Step 0 – see the landscape

Do:

```
uv run rdm story new-input --dhf dhf --list
```

This prints the bounded contexts, every taken DI id, the next free id, and the registered user needs — the vocabulary you need for every step below. An agent with the `rdm` MCP server (`.mcp.json`) can also `trace` a UN, DI or risk id, or `query` the record, read-only, before changing anything.

3.2 Step 1 – the user need (the WHY)

Why: validation anchors on user needs; a design input that refines no need is unexplainable, and the release gate blocks any need no input traces to.

Do: most changes refine an *existing* need — reuse it. Only a genuinely new validated journey gets a new UN-nnn, added to the `user_needs` frontmatter of `dhf/documents/verification_and_validation_plan.md` **plus** a row in that file's validation-approach table.

Done when: the need you'll cite appears in `--list` output above.

3.3 Step 2 – declare the design input (the WHAT)

Why: the design input is the verifiable requirement, the acceptance criterion the test will be judged against as a whole. Write each thing it requires so a test can check it; vague inputs produce unreviewable tests. What must be accepted on its own is a separate design input: write one `shall`, and never join with “and” two behaviours that could be accepted apart — a joined text invites a test of one half (Design Review 59).

Do:

```
uv run rdm story new-input --dhf dhf \
  --context <ctx> --text "RDM shall <what it must do>, ..." --traces-to UN-nnn
```

This allocates the next `DI-n`, inserts it into that context’s `design_inputs` frontmatter, writes a stub tagged test (*it fails on purpose* — see step 5), and prints your remaining checklist. Then, by hand, describe the intended output in that document’s `## Design Outputs`, naming the input by id. The frontmatter is the input’s only statement: the body never restates it. A context that helps realise an input owned elsewhere lists it under `realises` — an input is declared once, never duplicated.

Done when: `uv run rdm story trace DI-n` shows your input, its need, and its owning context.

3.4 Step 3 – commit the design docs FIRST (the approval)

Why: §820.30 requires design review before implementation; here the reviewed, committed doc is the approval. The pre-commit hook enforces the order: implementation commits are blocked while the design record is incomplete or uncommitted — but committing *only* design docs is always allowed (that’s how they become approved).

Do:

```
git add dhf/ && git commit -m "Approve design record: DI-n <what>"
```

Done when: `uv run rdm story design-gate --dhf dhf` prints PASSED.

3.5 Step 4 – implement (the HOW)

Do: build what the `## Design Outputs` prose names. If you discover the design was wrong, edit the design doc — the gate re-opens until the edit is committed. That friction is the feature: the record stays true.

3.6 Step 5 – make the test real (the PROOF)

Why: the design input is the acceptance criterion and the tagged test verifies it (“live BDD”) — there is no separate spec to drift out of date. The test’s verification steps are its own: one for each thing the design input requires, but never criteria of their own (a part that must be accepted separately is a separate design input). The scaffolded stub fails on purpose so the release gate

stays honestly red until real proof exists; a stub that passed would be a lie the pipeline could not see.

Do: replace the stub body in `tests/acceptance/` with real assertions against the real code path — **one verification step for each thing the DI text requires** — keeping the tag and naming the C4 component it exercises:

```
@allure.story("DI-n")                # the link the whole chain hangs on
@allure.label("component", "<c4 key>") # the component it exercises (dhf/c4/
workspace.dsl)
def test_<behavior>(...):
    """DI-n: <the requirement in one line>."""
    with verification_step("<what this step checks>"): # tests/acceptance/
        evidence.py
        result = ...
        attach("<what was checked>", result)          # kept in the Allure results
        assert ...
```

A named verification step makes a failure say what it checked; an attachment keeps what the assertion looked at. The steps belong to the test, never to the criterion. Declare only the story: `rdm.pytest_plugin` (hooked in `tests/acceptance/conftest.py`) labels every run from the record with Allure's API — epic (user need), feature (context), links to the Markdown that declares it (design document, V&V plan, risk document) at the commit, severity critical for a risk control, and the design input's text as an attachment. Never hand-write `@allure.feature/@allure.epic` for these. This is for acceptance (end-to-end) tests only: unit tests carry no Allure tags, steps or attachments. Both land in the Allure results, which the release bundle retains and the graph projects (`trace` lists them).

Changing an existing test: before adding a step to a test that already verifies a design input, read that input's text. If the text does not state what the new step checks, the step does not belong there yet: amend the text, or declare a new design input with its own test, in the same design review entry as the fix. Never let a test grow past its text — that is how DI-2's test reached fourteen steps under a text that named none of them (Design Review 59).

Done when: `uv run pytest tests/acceptance -q` passes, and each step checks something the input's text states.

3.7 Step 6 – run the gates as CI will

```
uv run rdm story design-gate --dhf dhf
uv run coverage run --source=rdm -m pytest tests --ignore=tests/acceptance -q #
unit tests, under coverage
uv run coverage xml -q -o dhf/data/unit-coverage.xml
uv run pytest tests/acceptance --clean-alluredir --alluredir=dhf/allure-results
uv run rdm story verify --dhf dhf --allure-results dhf/allure-results -o dhf/data/
verification.yml \
    --unit-coverage dhf/data/unit-coverage.xml
uv run rdm story release-gate --dhf dhf --allure-results dhf/allure-results
```

The unit tests' coverage is unit-test evidence the matrix shows beside the design inputs; it never verifies one (Design Review 63). Plus the general suite: `uv run pytest tests`, `uv run ruff check .`, and `uv run --extra docs mkdocs build --strict` if docs changed.

Done when: all four print PASSED (the matrix can then be rendered from the generated data: `uv run rdm render dhf/documents/traceability_matrix.md dhf/config.yml dhf/data/verification.yml` — generated output, never hand-edited).

3.8 Step 7 — commit, push, PR (the independent review)

Ordinary git from here. The pull request is the independent verification: a reviewer other than the author reads each DI's text against its tagged test and asks whether the test would fail if the behavior broke — a tautology, a mocked-out code path, or a test that checks two of the three things the DI requires is a reason to request changes. To check rather than eyeball, break one of them and run its test:

```
uv run rdm story mutation-probe --file <impl> --find '<code it requires>' \
  --replace '<one-line break>' --test <test_name> # KILLED = the test catches it
```

The reviewer says, in the review, that each changed design input's text was read beside its test (the pull-request template asks). Keep a fix pull request to one bounded context: no reviewer reads every input against its test inside a change that spans them all.

The merged, reviewed PR completes the approval record.

Merge with a merge commit — never squash or rebase. A squash folds the record-first commits (design document, then implementation) into one, so the history no longer shows the design was approved before the code; a rebase rewrites the commits that were reviewed. Configure the repository to allow merge commits only, as `examples/github-document-control/` does (`allow_squash_merge: false, allow_rebase_merge: false`).

4 Before a release — the chain review

No gate judges whether a test proves its design input; a person does. Before each release, read every design input with its whole chain — its user needs, its design prose, its tagged test and that test's verification steps, its component labels and its last run — and record what was found and decided as a design review, one input at a time (Design Review 58 is the first). Changes it asks for go through this loop like any other.

5 A worked example — from this repository's own history

`rdm story new-input` itself was added exactly this way; every artifact is in the repo to inspect:

Step	What happened	Where to look
1	UN-010 registered (“a contributor is guided to author a fully traced design input”)	<code>verification_and_validation_plan.md</code> frontmatter
2	DI-22 declared in the scaffolding context (since Design Review 30, specification), its text requiring six behaviours	<code>dhf/documents/design/specification.md</code>
3	Design docs committed <i>before</i> any code	<code>commit Approve design record: UN-010, DI-22, ...</code>
4–5	Implementation + tagged test	<code>rdm/gates/new_input.py</code> (since Design Review 32, <code>rdm/specification/new_input.py</code>), <code>tests/acceptance/test_scaffolding.py</code>
6–7	All gates green, pushed, PR reviewed	CI run on the PR

History: at the time, an independent review step (the since-retired faithfulness gate, Design Review 4) found that the test passed with a one-context fixture even though a mutant ignoring `--context` survived; the author strengthened the fixture to two contexts (commit `Strengthen the DI-22 test ...`). That kind of gap — a passing test that does not prove what its DI requires — is now the PR reviewer’s to catch.

6 Hard rules

Rule	Because
Never hand-edit the traceability matrix	it is generated from executed results; edits would be fiction
Never approve a PR you authored	self-review over-rates; independence is the entire point of the PR review
Design docs commit before implementation	the commit is the approval; the hook and CI enforce the order
Allure tags, steps and attachments go in acceptance tests only	unit tests are not evidence of record; <code>tests/allure_scope_test.py</code> holds the line

Rule	Because
Code coverage stays with the unit tests, never in Allure	acceptance evidence proves design inputs end to end; which lines ran is a unit-level measure
Merge PRs with a merge commit, never squash or rebase	a squash erases the design-before-code order the record-first commits prove
A DI is declared once; other contexts use <code>realises</code>	duplicated requirements drift apart
A test checks only what its DI's text states	a step the text does not state is a requirement nobody approved
One bounded context per fix PR	a reviewer cannot read every input against its test in one diff
A chain review, recorded as a design review, before each release	no gate judges whether a test proves its input
<code>dhf/allure-results/</code> , <code>dhf/data/</code> are generated, gitignored	evidence is produced by running, not by committing
Backlog/issues/plans are never cited as evidence	plan-vs-record boundary; the record is SDD + Allure + git
Backlog task files change only via the <code>backlog</code> CLI	see <code>AGENTS.md</code>
<code>RDM_SKIP_DESIGN_GATE=1</code> is for emergencies, and CI still runs everything	the bypass is loud and local-only

7 When a gate fails

Failure message (abridged)	It means	Fix
design-gate: <i>unresolved placeholders</i>	<code>TODO/ENDTODO</code> left in a design doc	finish the doc
design-gate: <i>uncommitted changes</i>	a design doc is edited but not committed	commit it (that commit is the approval)
pre-commit: <i>commit blocked</i>	implementation staged while the design gate fails	fix/commit the design record first
pre-commit: <i>'rdm' not on PATH ... blocked</i>	gate runner missing	<code>uv sync --all-extras</code> (elsewhere: <code>uv tool install "rdm @ git+https://</code>

Failure message (abridged)	It means	Fix
		<code>github.com/scope-impact/rdm";</code> PyPI's <code>rdm</code> is another project)
<i>release-gate: DI-n untested</i>	no executed result for the tag	write/tag the test, re-run the acceptance suite
<i>release-gate: DI-n failed</i>	the tagged test failed	fix the implementation (or the test)
<i>release-gate: user need addressed by no design input</i>	a UN nothing traces to	add a DI with <code>traces_to</code> , or remove the need
<i>release-gate: no risk_policy is declared</i>	the register has risks but no acceptability criteria	declare a <code>risk_policy</code> (see <code>docs/use/risk-register.md</code>); an agent's draft is <code>status: proposed</code>
<i>release-gate: risk ... residual not evaluated</i>	a controlling design input has no passing test	make that DI's tagged test pass
<i>release-gate: risk ... needs an acceptance / unacceptable residual</i>	the policy does not accept the residual as it stands	add or strengthen a control (a DI), or — where the policy says <code>justify</code> — record who accepted it and why
<i>release-gate: risk is proposed (warning)</i>	no person has approved that rating	a maintainer reviews it and sets <code>status: approved</code>
<i>orphan tag (warning)</i>	<code>@allure.story</code> id matches no declared DI	declare the DI or fix the tag